DEF CON 34 // LIVE DEMO

AgentHound

Inferred path to verified reachability

Isolated collector. Dashboard-first ingestion. Bounded retest. No host config exposure.

[THE PROMISE]

The finding ID stays unchanged. The campaign adds runtime evidence to the same CAN_REACH finding: unauthenticated control denied, authenticated exact-resource read allowed.

Prerequisites for a fresh attendee machine

Use macOS or Linux and run every host command from the AgentHound repository root.

[REQUIRED]

- A clean clone of the latest AgentHound repository. The demo Compose file, Dockerfiles, MCP service, and synthetic workstation fixtures are included.
- Docker Desktop or Docker Engine 24+ with Compose v2.20+; the Docker daemon must be running.
- Intel/AMD64 or Apple Silicon/ARM64, at least 4 GB RAM assigned to Docker, and 4 GB free disk.
- TCP port 18080 free on 127.0.0.1. Keep one host terminal open for the exported UID/GID variables.
- Internet access for the first pull/build only. Later clean runs can be offline.

[PREFLIGHT - COPY AND PASTE]

```
git clone https://github.com/adithyan-ak/AgentHound.git
cd AgentHound
export AGENTHOUND_DEMO_UID="$(id -u)"
export AGENTHOUND_DEMO_GID="$(id -g)"
test -f demo/defcon/compose.yml && \
  test -f demo/defcon/Dockerfile.workstation && \
  test -f demo/defcon/Dockerfile.mcp && \
  test -f demo/defcon/services/mcp-demo/main.go && \
  echo "Demo bundle present"
docker version
docker compose version
```

Keep this terminal open. This guide was validated against runtime commit bc06a9416d5a; use git checkout bc06a9416d5a only if a later main branch changes the checkpoints.

[OUTPUT CONTRACT]

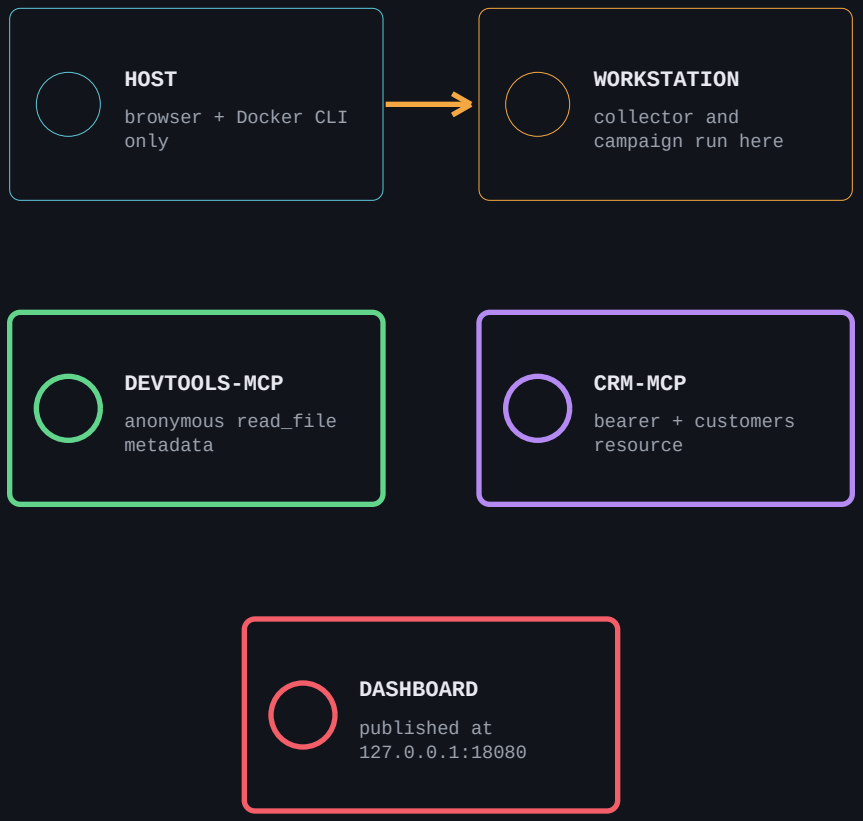
STABLE — Node/edge counts, artifact paths, finding states, confidence values, and campaign outcome must match this guide.

VARIABLE — Timestamps, scan UUIDs, campaign UUIDs, container suffixes, and the finding ID may differ. Copy your own finding ID when instructed.

Run the collector in the container, not on the host

The host runs only the browser and Docker CLI.

[ISOLATION MODEL]



Neo4j, PostgreSQL, and both MCP services stay on a private Docker network.

[WHY THIS PATH]

- The host's real Augment, Cursor, Claude, or MCP configuration is never scanned.
- Only synthetic configs are copied into /home/demo inside the workstation image.
- Only dashboard port 18080 is published. Databases and MCP services remain private.
- The first pull/build can happen before the lab. Repeat runs use local images.
- The reset command targets only the agenthound-defcon34 Compose project and volumes.

[TRUTH BOUNDARY]

The campaign verifies credential-gated reachability to the exact witness-bound resource. It does not prove credential acquisition, agent invocation, end-to-end chain execution, exfiltration, or impact.

03 // FIRST RUN - ONLINE

Pull runtime images, then build the three local images

Do this once while internet access is available. It is intentionally separate from startup.

COPY AND PASTE FROM THE REPOSITORY ROOT

```
# Pull the two runtime-only images
docker compose \
  -f demo/defcon/compose.yml \
  pull graph-db app-db

# Build the local product and fixture images
docker compose \
  -f demo/defcon/compose.yml \
  build devtools-mcp agenthound-server workstation
```

[EXPECTED END STATE]

```
graph-db Pulled
app-db Pulled

agenthound:defcon34-mcp Built
agenthound:defcon34-server Built
agenthound:defcon34-workstation Built
```

Docker may print cached layers, download progress, architecture-specific digests, and a UI bundle-size warning. Those are not failures if the command exits successfully and the five outcomes above appear.

[IF THIS PAGE FAILS]

- Cannot connect to Docker: start Docker Desktop/Engine, then repeat the command.
- No space left: free at least 4 GB in Docker's storage, then repeat.
- Pull/build network error: restore internet access; do not continue to --pull never yet.
- A frontend chunk-size warning is informational. Any non-zero command exit is a failure.

Reset only this lab, start from local images, then prove health

This is the repeatable day-of-lab start sequence. It can run offline after page 4 succeeds.

COPY AND PASTE

```
docker compose \
  -f demo/defcon/compose.yml \
  down --volumes --remove-orphans

docker compose \
  -f demo/defcon/compose.yml \
  up -d --wait --pull never

docker compose \
  -f demo/defcon/compose.yml ps

curl -fsS http://127.0.0.1:18080/api/v1/health
```

[GO / NO-GO]

- All six services report Up and healthy.
- agenthound-server shows 127.0.0.1:18080->8080/tcp.
- Health output is exactly: neo4j ok, postgres ok, status ok (JSON key order is not important).
- If --wait is unknown, upgrade Docker Compose to v2.20+.
- If port 18080 is already allocated, stop the conflicting process before retrying.

[EXPECTED HEALTH]

```
{"neo4j":"ok","postgres":"ok","status":"ok"}
```

Do not continue until this returns HTTP success. Startup normally takes 20-60 seconds on a fresh Docker installation.

Run the real collectors inside the workstation

Enter the isolated container once, collect two artifacts, then return to the host shell.

NATIVE SHELL + COLLECTOR COMMANDS

```
docker compose \
  -f demo/defcon/compose.yml \
  exec workstation bash

agenthound scan --config \
  --project-dir /home/demo/project \
  --output /demo/artifacts/config.json

agenthound scan --mcp \
  --project-dir /home/demo/project \
  --output /demo/artifacts/mcp.json
exit
```

[EXPECTED NATIVE OUTPUT]

```
Collected 11 nodes, 8 edges
output written path=/demo/artifacts/config.json
nodes=11 edges=8
Next: agenthound-server ingest
/demo/artifacts/config.json

Collected 7 nodes, 5 edges
output written path=/demo/artifacts/mcp.json
nodes=7 edges=5
Next: agenthound-server ingest
/demo/artifacts/mcp.json
```

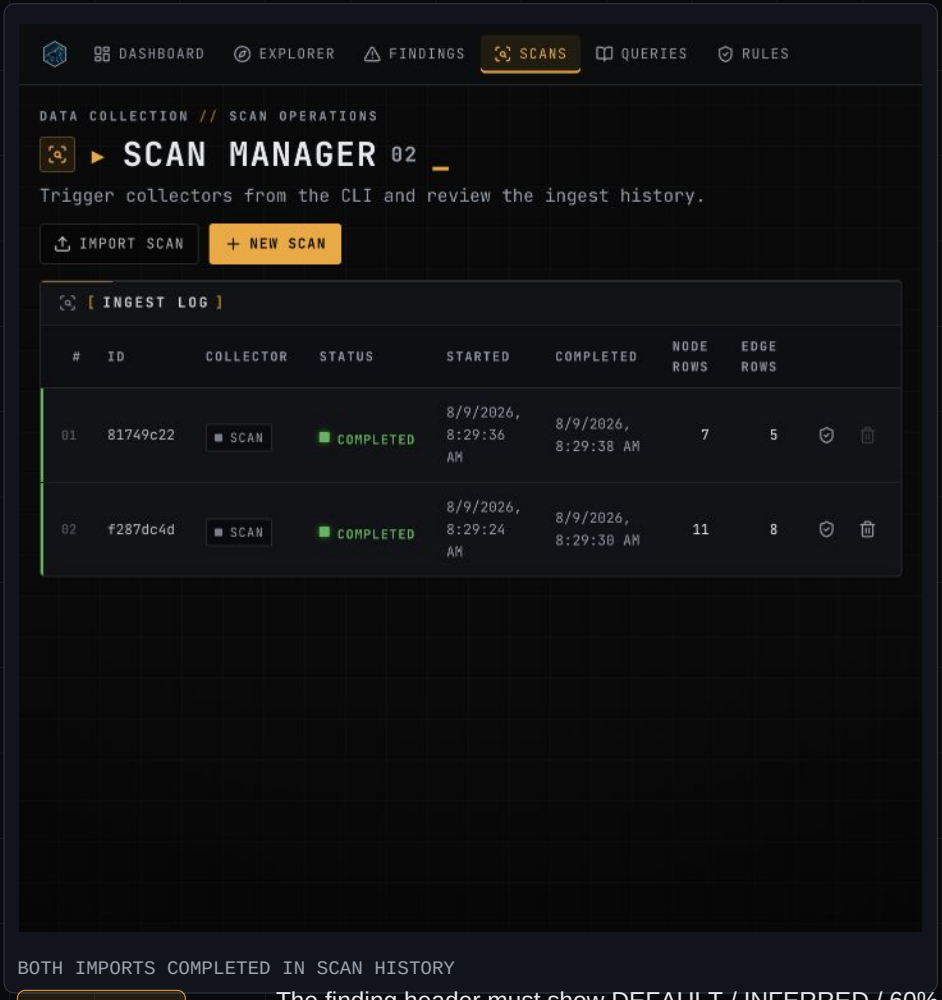
The CLI also prints timestamped rules-engine and collector log lines. Timestamps and container hostname vary. Stop if either node/edge count differs.

HOST FILES

The bind mount writes both JSON files into demo/defcon/artifacts on the host. Repeat runs overwrite those exact paths.

Import config.json, then mcp.json

Open http://127.0.0.1:18080. Do not use the CLI ingest suggestion printed by the collector.



BOTH IMPORTS COMPLETED IN SCAN HISTORY

[EXACT CLICKS]

01 Click Scans in the left navigation.

02 Click Import scan, then choose demo/defcon/artifacts/config.json.

03 Click Import and wait for the green success: 11 nodes / 8 edges.

04 Click Import another and choose demo/defcon/artifacts/mcp.json.

05 Click Import and wait for the green success: 7 nodes / 5 edges.

06 Click View findings.

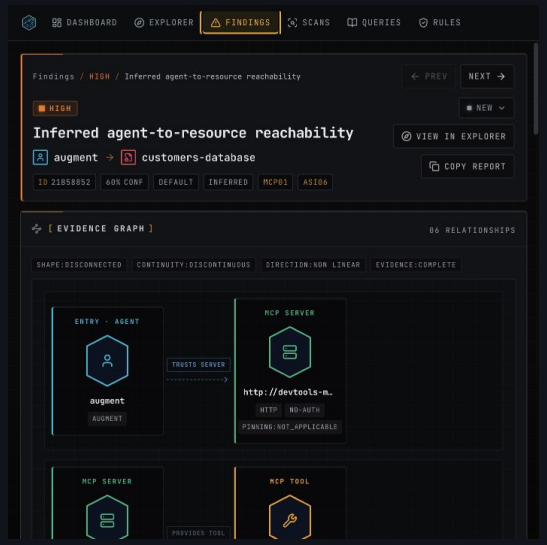
07 Select augment -> customers-database, severity HIGH.

CHECKPOINT

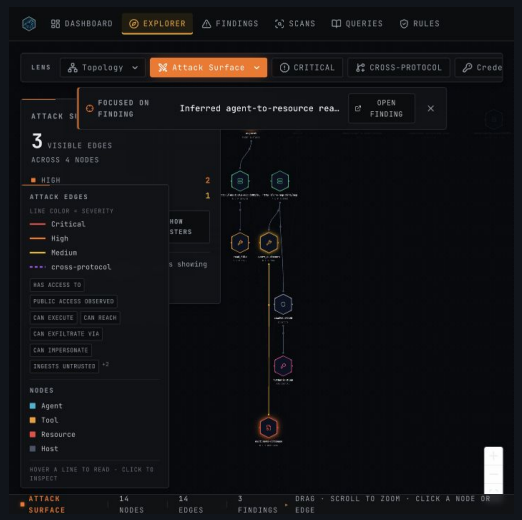
The finding header must show DEFAULT / INFERRED / 60% CONF. Three total findings are expected. Scan IDs and finding IDs can differ from the screenshots.

Explain the inferred path, then copy the full finding ID

The shortened header badge is not accepted by the witness command.



DEFAULT / INFERRED / 60 PERCENT



VIEW IN EXPLORER - ATTACK SURFACE LENS

[POINT IN THIS ORDER]

- AUGMENT: source agent and entry point.
- DEVTOOLS-MCP -> READ_FILE: anonymous server and credential_access + file_read tool.
- CREDENTIAL: hash-only hardcoded Authorization evidence; raw secret is absent from the graph.
- CRM-MCP -> QUERY_CUSTOMERS -> CUSTOMERS-DB: bearer-protected target tool and exact high-sensitivity resource.

[COPY THE ID]

- Return to the finding detail if Explorer is open.
- Scroll to the References section at the bottom.
- Copy the full 16-character Finding ID. Do not copy the 8-character badge shown near the title.
- Keep the browser open on this finding; the same full ID must remain after the campaign import.

Export the native witness for the selected finding

Run these commands in the host terminal from the repository root, not inside workstation.

COPY AND PASTE

```
printf 'Paste full Finding ID: '; IFS= read -r FINDING_ID
# Paste the 16-character ID and press Enter.

docker compose \
  -f demo/defcon/compose.yml \
  exec -T agenthound-server \
  agenthound-server witness \
    --finding "$FINDING_ID" \
    --output - \
  > demo/defcon/artifacts/witness.json

test -s demo/defcon/artifacts/witness.json && \
  echo "Witness ready: demo/defcon/artifacts/witness.json"
```

[EXPECTED CHECKPOINT]

```
Witness ready:
demo/defcon/artifacts/
witness.json
```

Connection, schema, or driver log lines from agenthound-server may appear in the terminal because they are written to stderr. That is normal if the Witness ready line appears.

[WHAT THE WITNESS BINDS]

- The selected source agent, credential evidence, target MCP server, and exact customers resource.
- The current published graph revision. Do not reset or re-import config/mcp between witness export and campaign.
- If the command says finding not found, repeat page 8 and copy the full ID from References.

Run the bounded credential-reach campaign

This is the actual product command; no demo wrapper is used.

COPY AND PASTE

```
docker compose \
  -f demo/defcon/compose.yml \
  exec -T workstation \
  agenthound campaign \
    http://crm-mcp:8931/mcp \
    --scenario cred-reach \
    --witness /demo/artifacts/witness.json \
    --credential-env AGENTHOUND_CAMPAIGN_CREDENTIAL \
    --engagement-id DEFCON34-CRED-REACH \
    --commit \
    --output /demo/artifacts/campaign.json
```

[EXPECTED NATIVE OUTPUT]

```
[campaign] RUN_REPORT {...}
[campaign] COMMITTED cred-reach on
http://crm-mcp:8931/mcp
outcome=credential_gated_reach_verified
control=denied authed=allowed
output written path=/demo/artifacts/
campaign.json nodes=2 edges=1
Next: agenthound-server ingest
/demo/artifacts/campaign.json
```

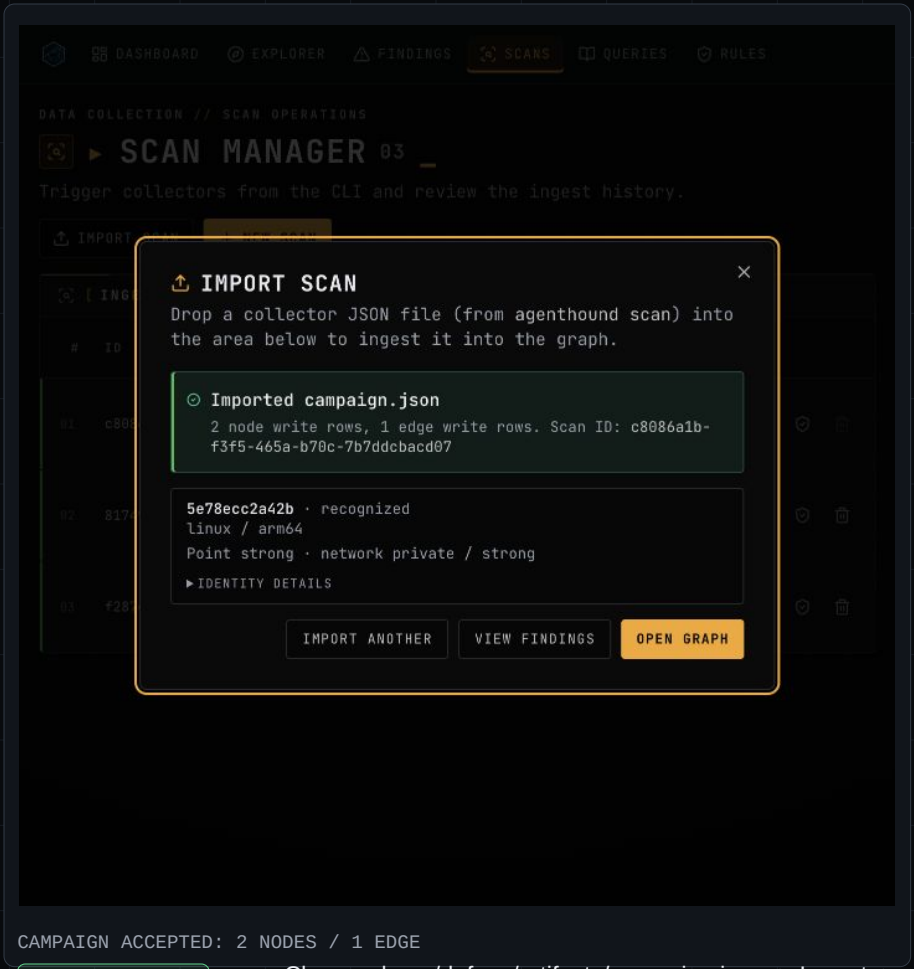
RUN_REPORT is one long JSON line. Its IDs and timestamps vary; the outcome, denied/allowed differential, path, and 2/1 counts above must match.

[SAFETY AND MEANING]

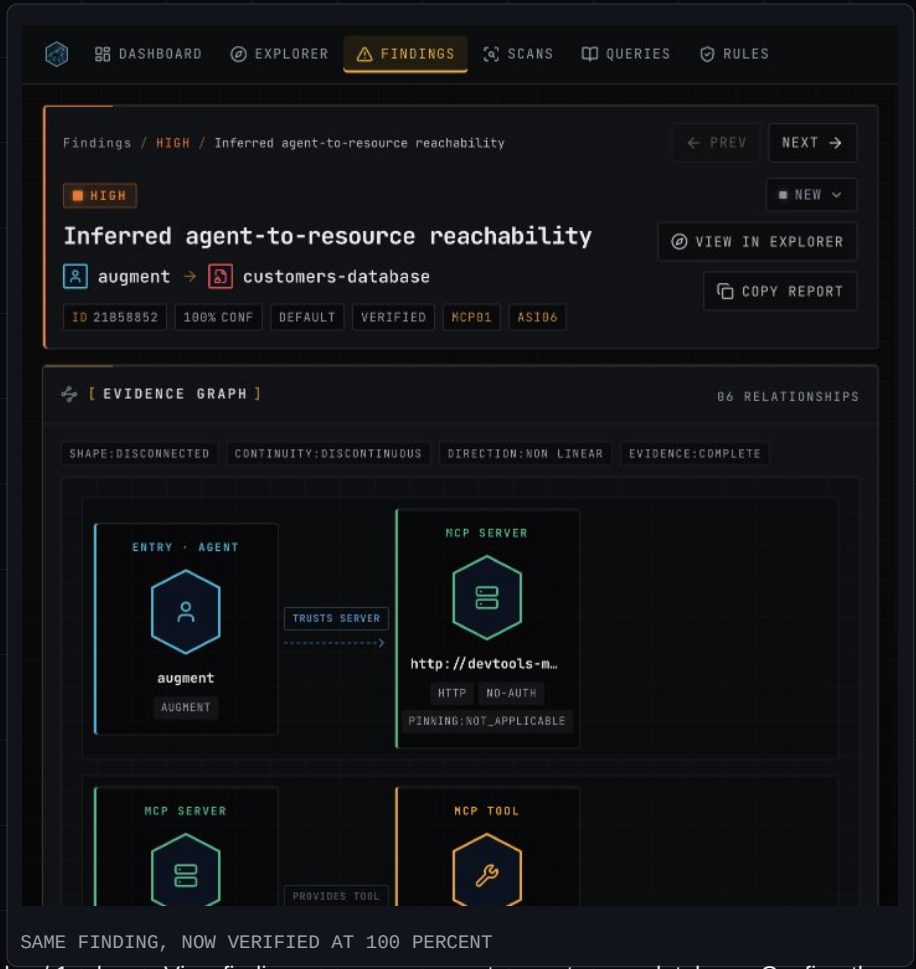
The control probe initializes without the credential and is denied without addressing the resource. The authenticated probe performs a read-only exact-resource read and is allowed. The request budget is bounded, the mutation limit is zero, and cleanup is not applicable.

Import campaign.json and reopen the original finding

Dashboard: Scans -> Import scan. This is the third and final browser import.



CAMPAIGN ACCEPTED: 2 NODES / 1 EDGE



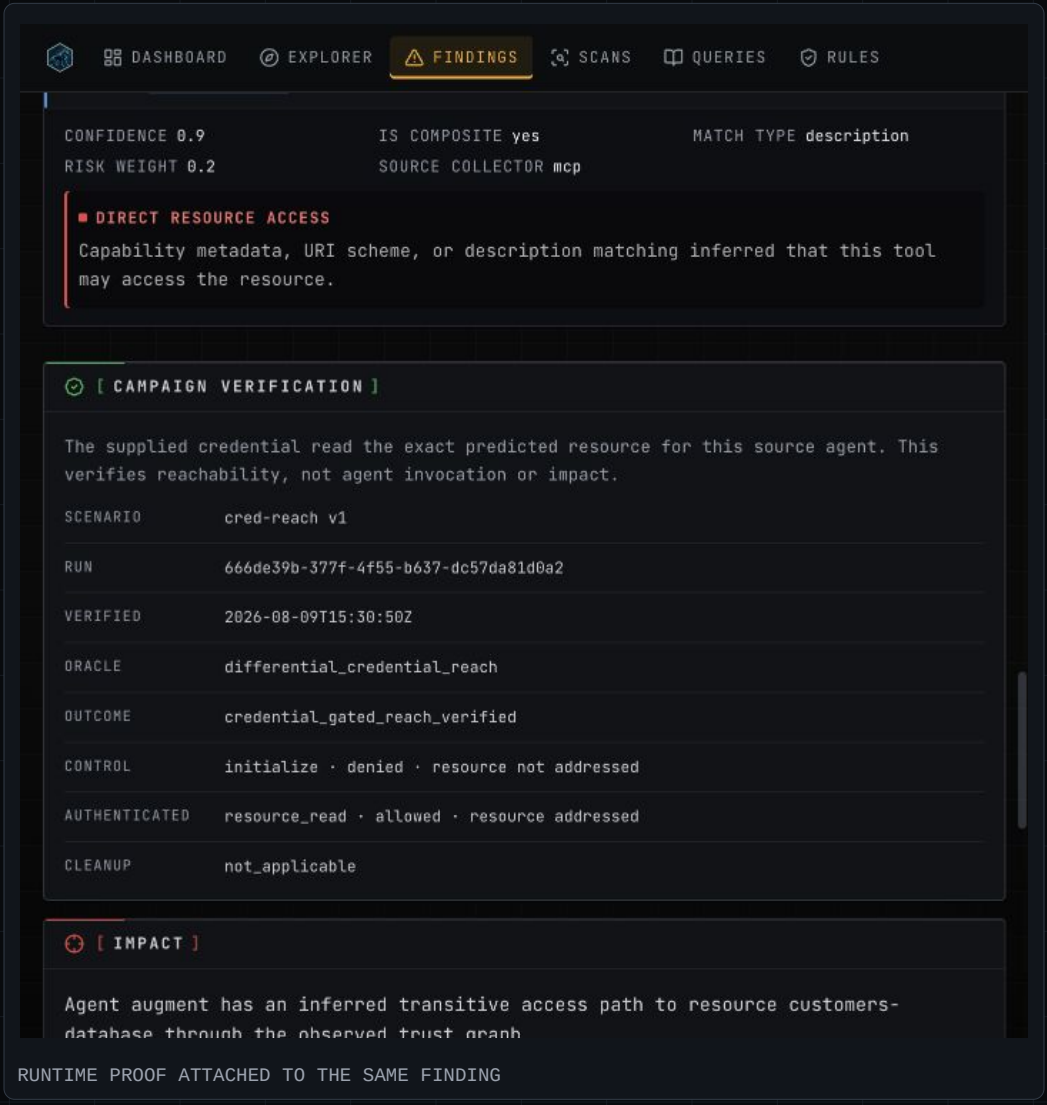
SAME FINDING, NOW VERIFIED AT 100 PERCENT

EXACT CLICKS

Choose demo/defcon/artifacts/campaign.json -> Import -> wait for 2 nodes / 1 edge -> View findings -> open augment -> customers-database. Confirm the full ID from References is unchanged, and the header now shows DEFAULT / VERIFIED / 100% CONF.

Show the Campaign Verification panel

Scroll below Relationship Evidence on the verified finding.



RUNTIME PROOF ATTACHED TO THE SAME FINDING

[READ THE DIFFERENTIAL]

OUTCOME

credential_gated_reach_verified

CONTROL

initialize / denied / resource not addressed

AUTHENTICATED

resource_read / allowed / resource addressed

Conclusion: possession of the bound credential changes the observed outcome for the exact predicted resource. This verifies reachability for this source-agent topology.

Do not claim prompt-driven execution, credential theft, exfiltration, or customer impact.

If a checkpoint differs, stop and repair it here

Do not improvise past a failed count, health check, import, or finding state.

[COMMON FAILURES]

- Compose says to set AGENTHOUND_DEMO_UID/GID: repeat the two export commands on page 2 in the current terminal.
- Image pull/build fails: restore network, repeat page 4, and require a zero exit status.
- Port 18080 already allocated: stop the conflicting process, then repeat the clean start on page 5.
- Dashboard health fails: inspect with the page 5 compose ps command; all six services must be healthy.
- Import has the wrong counts: verify the selected filename and recollect. Import config first, then mcp, waiting for each green success.
- No inferred 60% finding: reset and repeat pages 5-7 in order. Do not import a host-generated artifact.
- Witness says finding not found: use the full 16-character ID from References, not the 8-character header badge.
- Campaign does not verify: export a fresh witness after the two initial imports; do not reset before the campaign import.

[NORMAL SHUTDOWN]

```
docker compose \
  -f demo/defcon/compose.yml down
```

This stops the stack but retains database volumes. Artifact JSON files remain in demo/defcon/artifacts.

[FULL LAB RESET]

Use the page 5 down --volumes --remove-orphans command. It clears only this demo's database volumes. Collector commands overwrite the three imported artifact paths on the next run.

[FINAL SUCCESS CONTRACT]

Three imports: 11/8, 7/5, 2/1. One augment -> customers-database finding: DEFAULT / INFERRED / 60% before, same full ID at DEFAULT / VERIFIED / 100% after. Campaign outcome: credential_gated_reach_verified, control denied, authenticated allowed.